

NAVIS

The Planning-to-Execution Bridge for Infrastructure Projects

Final Research & Engineering Report

Smart India Hackathon 2026 · Problem Statement 26122 · Intelligent Data
Capture & Schedule-Linking Layer

Evidence discipline	every measured number traces to a runnable harness in research/data/
Component statuses	audited against code — research/NAVIS_TECHNICAL_AUDIT.md
Rubric content	clearly labelled as judgement, never presented as measurement

Self-contained: this report can be read without opening the source code. Repository evidence is cited in every section.

Contents

1. Executive summary	4
2. SIH problem statement	4
3. The real-world problem	4
4. NAVIS solution overview	5
5. Current system architecture	5
6. Complete end-to-end data flow	6
7. Data ingestion pipeline	7
8. Extraction pipeline	8
9. Schedule-linking / matching engine	8
10. Hybrid retrieval explanation	9
11. Confidence scoring and reconciliation	9
12. Human-in-the-loop workflow	9
13. High-confidence vs low-confidence handling	10
14. Unmatched / new activity handling	11
15. Database and data lifecycle	11
16. Security architecture	12
17. Prompt-injection protection	12
18. Audit trail	12
19. Institutional memory	12
20. Analytics	13
21. Competitor analysis	13
22. Innovation analysis	14
23. Technical defensibility	15
24. PS requirement coverage	16
25. Current MVP completeness	16
26. Scalability	17
27. MVP architecture vs production architecture	17
28. Experiments / evaluation results	18
29. Weaknesses and limitations	19
30. Recommended improvements	20
31. SIH judge analysis	20
32. Demo strategy	21

33. Difficult judge questions and answers	22
34. Final verdict	23
35. The strongest one-line description of NAVIS	23
Appendix — evidence index	24

1. Executive summary

NAVIS is a working planning-to-execution bridge for infrastructure projects. It ingests heterogeneous field inputs — free-text daily progress reports (DPRs), discipline spreadsheets, and a conversational supervisor agent — extracts activity-level actual start/finish events with per-event provenance, links them to L5/L6 schedule activities with a measured confidence score, and writes the schedule back with a full append-only audit trail. Anything it is not sure about goes to a planner review queue instead of the schedule.

The matching engine is **measured**, not claimed: on 242 gold-positive mentions against a 120-activity schedule it reaches **87.2% Top-1 accuracy**, and at the shipped operating point ($\tau_{\text{high}} = 0.775$) it auto-links **50.4% of all mentions with 100.0% precision — zero wrong auto-links**. A wrong auto-link corrupts the schedule; a review item only costs a planner about 10 seconds. The policy is calibrated on that asymmetry, and the precision/coverage curve behind it is measured, not asserted.

Headline measured results (eval.py — 254 labelled mentions)

Top-1 accuracy **87.2%** · auto-link precision **100.0%** at 50.4% coverage · suggestion precision 83.1% / recall 85.5% · end-to-end latency 266 events in 1.86 s (~7 ms/event) · cold start 4.4 s once per process. Ablations, the BM25-vs-hybrid gate curve and the dense-feature fix are all reproduced from research/data/.

Institutional memory — the half of the problem statement competitors skip — is implemented: actual-vs-planned durations, discipline slip, recurring delay causes and suggested durations, computed live from captured execution data with sample sizes shown rather than hidden. Equally important is what NAVIS does *not* claim: there is no OCR, no Primavera import (export only), no authentication, and the alias learning loop is write-only today. These limitations are named here and in every summary chart, because a claim that survives a judge's probe is worth more than one that does not.

2. SIH problem statement

PS 26122 — Intelligent Data Capture & Schedule-Linking Layer for Infrastructure Project Management: Real-Time Actual Progress Tracking (Planning-to-Execution Bridge). Infrastructure schedules cascade from macro milestones (L1) down to micro, executable activities (L5/L6) across civil, piping, equipment, electrical, instrumentation and HSE disciplines. The baseline plan is well-structured in Primavera/MS Project, but actual execution flows back through daily progress reports, site diaries, discipline spreadsheets and verbal supervisor updates — each in its own format, largely disconnected from the L5/L6 activity IDs in the plan.

The PS asks for five things: (1) ingest heterogeneous inputs and extract activity-level actual start/end events; (2) an LLM-based conversational or voice 'time agent' for site supervisors; (3) fuzzy-match extracted descriptions to the correct L5/L6 node, handling terminology and granularity mismatches, flagging unmatched/new activities rather than dropping them; (4) auto-update actual dates with a confidence score and audit trail per entry; and (5) a clean, discipline-tagged progress dataset feeding both live analytics and a queryable institutional-memory repository. A prototype demonstrating 2–3 input formats, extraction and schedule-linking is deemed ideal; full production OCR/ASR is explicitly *not* required.

3. The real-world problem

On a real EPC site, the plan and the truth about the plan live in different worlds. The schedule is precise: *“Erect Line 24”-P-1001-A1A, Unit 2 rack, planned 2026-03-04 → 03-09, 120 M”*. The field says: *“spool erection on the 24 inch header is done, 6 nos”* — spoken, not typed; mangled by speech recognition; with no activity ID, no agreed date format, and no agreed unit.

- **Fragmentation.** Each discipline reports in its own format and cadence; the same physical progress is described differently by piping, civil and electrical crews.

- **Latency.** Manual reconciliation with the baseline lags the schedule update cycle by days or weeks, so every downstream analysis inherits stale data.
- **Granularity mismatch.** Field work is often more granular than the planned WBS: many field mentions roll up into one L5/L6 node.
- **Lost knowledge.** When a project closes, real durations, bottlenecks and delay causes stay locked in supervisors' heads and paper records — not queryable.
- **Asymmetric error cost.** A missed update is an annoyance; a wrong actual date silently corrupts the schedule and every forecast built on it.

4. NAVIS solution overview

NAVIS is one FastAPI service plus a two-role React UI, organised around a single spine: **every schedule-relevant fact carries its provenance, its confidence and its audit trail**. Two roles share one database: the planning engineer ingests files and adjudicates the review queue; the field supervisor logs progress by talking or typing.

- **Ingest** — POST /ingest accepts DPR text, spreadsheets and agent turns; byte-identical content is refused by sha256; each upload becomes a job record.
- **Extract** — a deterministic regex pre-pass owns tags, dates, quantities and units; an optional LLM (off by default) may only read intent out of informal prose.
- **Link** — hybrid retrieval (exact tag + BM25 + dense MiniLM, fused by RRF) feeds a 6-feature scorer and a calibrated decision policy: AUTO_LINK, REVIEW, NEW_ACTIVITY.
- **Reconcile** — planners see every suggestion with its evidence and one-key confirm; clarifications can be sent back to the supervisor who raised the item.
- **Remember** — every decision appends to an immutable audit trail and feeds the institutional-memory queries: durations, delays, productivity, suggested durations.

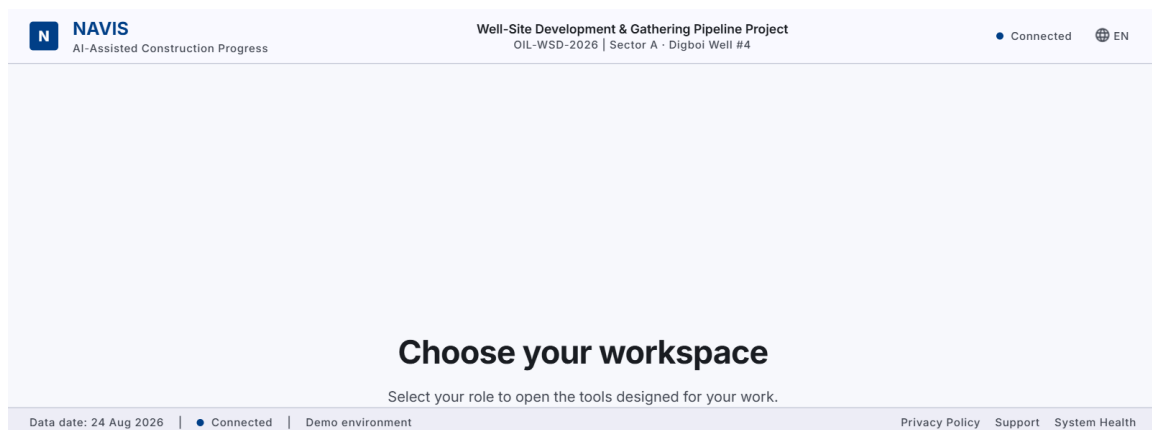
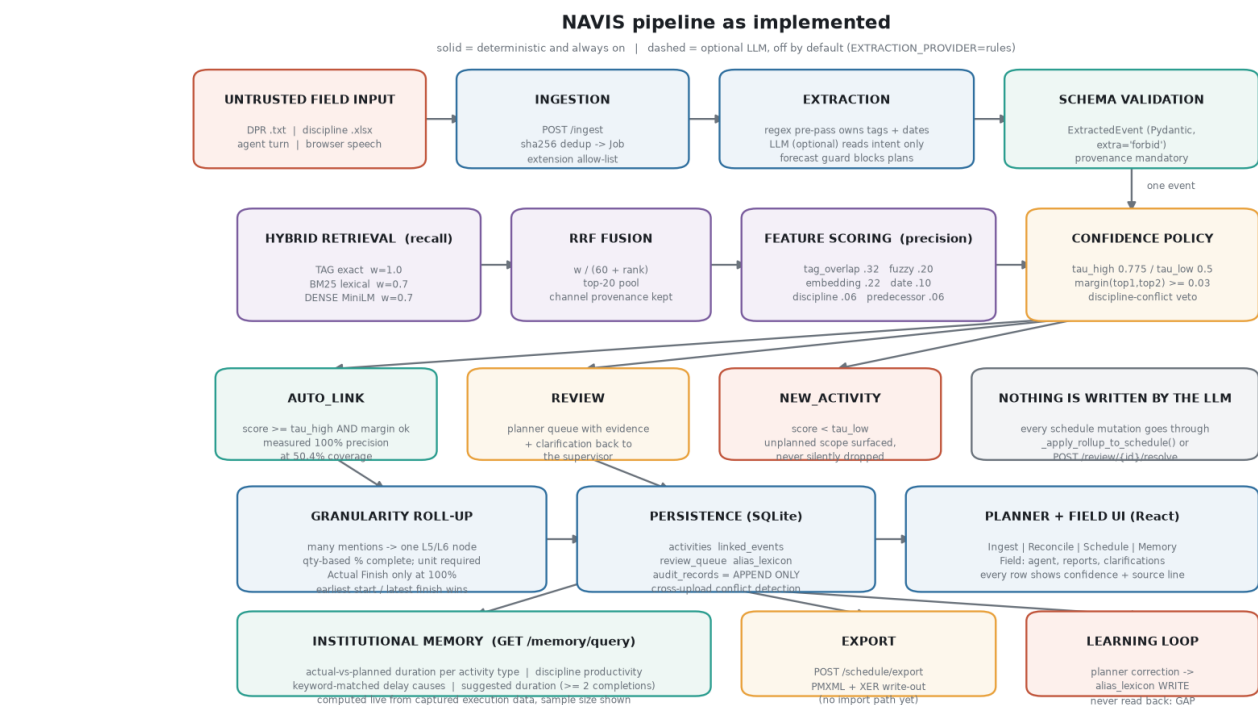


Figure 1. Role selection — the entry point of the two-role React UI (planning engineer / field supervisor). Design mockup from Design/; illustrative data, not a live screenshot.

5. Current system architecture

The architecture keeps **retrieval and ranking deliberately separate**: retrieval is recall-oriented (cast a wide top-20 net), ranking is precision-oriented (feature scoring). Merging both into one opaque LLM call would destroy explainability and the audit trail. Solid stages in Figure 1 are deterministic and always on; the dashed LLM is optional and off by default (EXTRACTION_PROVIDER=rules).



AUDITED - drawn from matching/, extraction/, server/ and frontend/src as they exist on this branch. Threshold and coverage figures are measured (research/EXPERIMENTS.md).

Figure 2. NAVIS pipeline as implemented — audited against matching/, extraction/, server/ and frontend/src on this branch (AUDITED + MEASURED). Note the three decision outcomes, the “nothing is written by the LLM” rule, and the red learning-loop box: the alias lexicon is written but never read back — a named gap.

Two structural choices matter for the demo story: **retrieval and ranking are separate stages on purpose** (separating recall from precision keeps every match explainable), and **the LLM never touches the schedule** — every schedule mutation flows through the deterministic roll-up or a planner resolution endpoint.

6. Complete end-to-end data flow

One DPR sentence — “Erection of 24"-P-1001-A1A completed 6 out of 18 spools” — traces through the system as follows (each step names its code location):

- **1 • Ingest.** POST /ingest stores the file, sha256-dedups byte-identical content, creates a Job row (server/main.py).
- **2 • Pre-pass.** Regex extracts the tag 24"-P-1001-A1A, the year-less date (resolved against the report's own header date), the fraction 6/18 and the unit — each with its character span (extraction/prepass.py).
- **3 • Event.** An ExtractedEvent is formed: status FINISH, discipline PIPING, provenance = file + line + span (extraction/extractor.py, models.py).
- **4 • Retrieval.** The tag channel, BM25 and MiniLM each propose candidates; weighted RRF fuses them into a top-20 pool, keeping per-channel provenance (matching/retrieval.py).
- **5 • Scoring.** Six features per candidate — tag overlap, discipline agreement, date proximity, predecessor plausibility, fuzzy similarity, embedding cosine — blend with renormalised weights (matching/features.py).
- **6 • Decision.** AUTO_LINK if score ≥ 0.775 with margin ≥ 0.03 and no discipline conflict; NEW_ACTIVITY if score < 0.5 ; REVIEW otherwise (matching/engine.py).
- **7 • Roll-up.** Mentions accumulate on the node: 6/18 = 33% complete; Actual Finish is withheld until 100%; earliest start / latest finish wins; disagreements are recorded, not silently resolved (matching/engine.py).
- **8 • Persist + audit.** The schedule row updates; an append-only AuditRecord stores field, old→new, source file + line, confidence, and whether a planner confirmed it.

7. Data ingestion pipeline

The seeded corpus is exactly what the PS recommends: eleven DPR text files (including one deliberately messy file, `dpr_day_11_messy.txt`) and two discipline spreadsheets — 13 files, 266 events. Ingest is synchronous: the PS’s “near real time” is delivered as **on-submission**, with no queues or workers to fail on stage.

Input	Handler	Key behaviours
DPR free text (.txt)	extraction/textio.py → extractor.py	encoding chain utf-8-sig → cp1252 → latin-1 (zero U+FFFD in the database); year-less dates resolved against the report header date, ambiguous beyond ±183 d flagged
Discipline spreadsheets (.xlsx)	extraction/spreadsheet.py	merged-header tolerant; column semantics mapped per discipline; row-level provenance
Conversational agent turn	server/agent_slots.py	slot-filling for discipline, location, status, date, completed + planned quantity; every value re-validated by deterministic parsers
Browser speech-to-text	frontend/src/hooks/useSpeech.ts	Web Speech API with transcript review; typed fallback verified end-to-end
Scanned diaries / OCR	—	NOT FOUND in the repo (PS waives production OCR)

Table 1. Ingestion surfaces and their guarantees.

NAVIS

HOME

RECONCILE

SCHEDULE

INGEST

MEMORY

Data Date

2026-09-15

PD

Priya Das

PLANNING ENGINEER

SIGN OUT

OIL Well-SiteDuliajan / OIL-WSD-2026 / Sector A · Digboi Well #4

INGEST Project Updates

Upload structured site reports for extraction and human review.

Drop a .txt or .xlsx file here

BROWSE FILES

Accepted formats: daily progress reports (.txt) and discipline spreadsheets (.xlsx). Do not accept photos, images, PDFs or scanned documents.

PIPELINE TRACE

PARSED

dpr_day_03.txt · 2,151 bytes

EXTRACTED

15 progress events

PROPOSED

10 high-confidence candidates · 5

require review

STAGED

15 events recorded · no actual dates written before approval

EXTRACTED EVENTS (DPR_DAY_03.TXT)

FILTER

COMMIT VALIDATED

SRC LN	RAW TEXT (SNIPPET)	DISCIPLINE	TAGS	DATE	CONF.
0042	Completed poured concr...	CIVIL	PAD_A1, C_POUR	26-09-14	0.98
0045	Installed rebar grid s...	STRUCT	REB_B2, I_GRID	26-09-14	0.95
0051	Pump station wiring 50...	ELEC	PUMP_W, AMBIG	26-09-14	0.45
0055	Replaced broken valve ...	MECH	VLV_THP, REP	26-09-14	0.88
0062	Tested pressure on mai...	QAQC	MAIN_L, P_TEST	26-09-14	0.91

RECENT INGEST HISTORY

VIEW ALL

dpr_day_02.txt	2026-09-13 18:45	12 events	12 confirmed
structural_wk3.xlsx	2026-09-12 09:12	45 events	42 confirmed

Figure 3. Planning engineer — Ingest screen. Design mockup; the shipped UI follows this layout (frontend/src/pages/Ingest.tsx).

NAVIS — SIH 2026 Final Report · PS 26122

Page 7

8. Extraction pipeline

The extraction design answers one question well: **what is the LLM actually for?** Regex is excellent at tags, dates, quantities and units — so a deterministic pre-pass owns all of them, with character-span provenance on every fragment. The LLM is used only where regex is weak — reading intent out of informal prose — and is structurally prevented from inventing the fields that reach the schedule: it is never asked for dates or tags-as-truth (extraction/llm_backend.py).

Extraction stage	Owner	Notes
Tags (pipe/equipment/instrument)	regex pre-pass	24"-P-1001-A1A, V-101, PSV-01 patterns; feed the near-decisive tag_overlap feature
Dates (explicit, ISO, year-less, relative)	regex pre-pass	year-less dates resolved against the report's own header date; ambiguous ones flagged, not guessed
Forecast-vs-actual guard	regex + extractor	planned/forecast language can never write an actual date; tested in extraction/test_llm_guards.py
Quantities, units, percentages	regex pre-pass	"6 out of 18" stored as completed 6, planned 18 — the denominator decides completion
Discipline + status intent	LLM (optional)	enum-constrained output; off by default; deterministic inference when off
Provenance	all stages	file + line/row + character span on every event

Table 2. Extraction responsibilities: deterministic where the data is structured, LLM only for intent.

Prompt-injection posture (extraction path)

The LLM sees prepass-enriched text but its output is schema-validated (Pydantic, extra='forbid'), enum-constrained, and re-validated by the same deterministic parsers before use; it supplies no dates and no tags. A malicious DPR can at worst distort intent classification — the fields that reach the schedule remain regex-owned.

9. Schedule-linking / matching engine

The engine (matching/engine.py) consumes ExtractedEvents and returns a LinkDecision with a ranked candidate list, per-channel retrieval sources, per-feature scores and a written rationale. The decision rule is precision-first:

Outcome	Condition	Consequence
AUTO_LINK	score ≥ 0.775 AND margin(top1, top2) ≥ 0.03 AND no discipline conflict	written to the schedule automatically; measured 100% precision
REVIEW	everything in between — including a high score with an ambiguous margin	planner queue with evidence; ~10 s per item; clarification can be sent back to the field
NEW_ACTIVITY	top-1 score < 0.5	unplanned scope surfaced for the planner — never silently dropped

Table 3. The three-outcome decision policy (matching/models.py Thresholds: tau_high=0.775, tau_low=0.5, margin_min=0.03).

A domain-specific rule sits at decision time: **the line lock**. A full line+size+spec tag match resolving to a unique schedule activity floors the score at 0.93 — the line number pins the node almost by itself — and any line-locked candidate floors just above tau_low, so the engine can never call identified-scope work NEW_ACTIVITY (matching/features.py, blend_with_line_lock).

10. Hybrid retrieval explanation

Three channels retrieve a candidate pool; weighted reciprocal-rank fusion ($k=60$) combines them into a top-20 set. The tag channel dominates deliberately: a matching line number is near-decisive evidence and must anchor the pool.

Channel	Weight	Role	Measured
TAG (exact tag)	1.0	anchor: line numbers resolve activity identity	alone: Top-1 only 4.9% — tags identify, they don't rank
BM25 (lexical)	0.7	terminology overlap: 'spool erection' ↔ 'Erect Line...'	alone: Top-1 93.8%, best ranker
DENSE (MiniLM, local)	0.7	paraphrase beyond shared words	alone: Top-1 88.8%; R@20 100%
Fusion (RRF $k=60$)	—	top-20 pool with per-channel provenance	R@20 100% (ablation.json)

Table 4. Retrieval channels and their measured contributions (research/data/ablation.json).

11. Confidence scoring and reconciliation

Confidence is a weighted blend over the features that are actually present; absent signals are excluded and the weights renormalise rather than penalising (None means 'no evidence', not 'evidence against'). Discipline mismatch scores 0.3, not 0 — an inference error must not nuke a true match.

Feature	Weight	What it encodes
tag_overlap	0.32	exact/near-exact tag match; line+size+spec sets line_locked
embedding_cosine	0.22	MiniLM cosine similarity
fuzzy_similarity	0.20	rapidfuzz token-set ratio vs description and extended description
date_proximity	0.10	reported date inside the planned window (+3/−10 d grace), exponential decay beyond
discipline_agreement	0.06	soft signal — inference from field text is noisy
predecessor_plausibility	0.06	is the activity logically startable at the reported date?
line lock (decision-time)	floor 0.93 / 0.46	unique line match / any line-locked candidate

Table 5. Feature weights (matching/features.py FEATURE_WEIGHTS) and the line-lock rule.

Reconciliation across sources is governed by explicit rules rather than silence: **earliest start wins, latest finish wins, an explicit assertion beats a bare reported date**; quantity-based roll-up writes Actual Finish only at 100% of planned quantity (a partial-scope finish assertion is withheld and reported); disagreements between sources are recorded as conflicts for the planner (25 conflicts surfaced on the seeded corpus, 21 of them spreadsheet-vs-DPR).

12. Human-in-the-loop workflow

The review queue is the system's safety valve and its learning surface. Items land there with full evidence: the source text with the matched span highlighted, the extracted entities, the ranked candidate activities with confidence, and the review reason. The planner confirms, reassigns, flags unscheduled work, or rejects — and can send a clarification back to the supervisor who raised the item, closing the loop at the field level.

Project Alpha
Site 21B - Phase 2

NAVIS

OIL-WSD-2026
Sector A - Digboi Well #4

Priya Das
Planning Engineer

Home

Reconcile

Schedule

Ingest

Memory

113 PENDING

All Disciplines

Only below 60%

MECHANICAL

54% CONF P1

"Installed 30m of 4in CS pipe at Sector A, waiting on..."

ELECTRICAL

82% CONF P2

"Pulled 500m power cable to well pad 4, termination ..."

CIVIL

41% CONF P1

"Poured foundation for pump skid, slight delay due to..."

SOURCE EVIDENCE

FR-2023-10-24.txt : L42

...morning toolbox talk completed. Weather clear.
Installed 30m of 4in CS pipe at Sector A, waiting on NDT.
Crew demobilized at 1700 hrs. Minor incident reported...

Review Reason: Low confidence (54%) matching '4in CS pipe' to schedule activity. Multiple potential matches found.

EXTRACTED ENTITIES

DISCIPLINE

Mechanical

EQUIPMENT / MATERIAL

4in CS pipe

QUANTITY

30 m

ASSERTED DATE

2023-10-24

CONFIDENCE SCORE

0.54

CANDIDATE ACTIVITIES

1 ACT-M-4012

Planned: 2023-10-20 to 2023-10-28

Install 4" Carbon Steel Piping - Sector A
Discipline: MECH Planned Qty: 150 m

2 ACT-M-4015

Planned: 2023-10-25 to 2023-11-02

Tie-in 4" CS Spools - Manifold Area
Discipline: MECH Planned Qty: 20 ea

Confirmed. 2 alias entries created · 3 audit records appended.

REJECT

FLAG AS UNSCHEDULED WORK

CONFIRM MATCH

↑/↓ or J/K Navigate · 1-9 Select · Enter Confirm

U Unscheduled · R Reject

Figure 4. Planning engineer — Reconcile screen with source evidence, extracted entities and candidate activities. Design mockup (illustrative data such as 'ACT-M-4012' and '113 PENDING' are from the design phase; the shipped queue in the seeded run holds 118 items).

13. High-confidence vs low-confidence handling

The threshold is a measured operating point, not a guess. Raising τ_{high} trades coverage for precision; 0.775 is the first point where a wrong auto-link stops happening entirely — at the cost of pushing more items to review. Below it, coverage climbs (72.4% at 0.55) but precision drops to 92.4%: the schedule starts absorbing errors.

NAVIS — SIH 2026 Final Report · PS 26122

Page 10

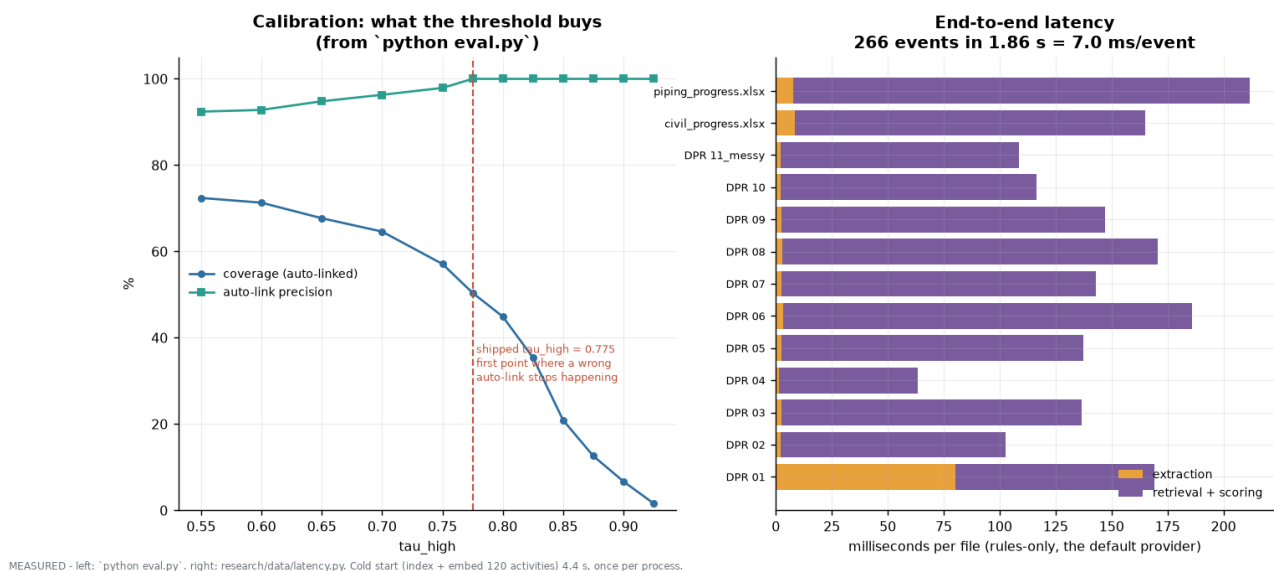


Figure 5. Left: precision-at-coverage sweep of `tau_high` from `eval.py` — the shipped `tau_high=0.775` (MEASURED). Right: end-to-end latency per file, extraction vs retrieval+scoring (research/data/latency.py, MEASURED); 266 events in 1.86 s, cold start 4.4 s once per process.

High-confidence items never touch a human; low-confidence items cost a planner about 10 seconds each (121 review items on the seeded corpus, ~20 minutes of planner time for 254 mentions). The demo rehearsal resets to this exact state with `scripts/demo_reset.ps1` and verifies the summary line before the run.

14. Unmatched / new activity handling

Work that does not belong to any planned node is a finding, not noise. `NEW_ACTIVITY` surfaces it to the planner (who can create an activity or discard it explicitly); `REVIEW` items can be flagged as unscheduled work. Nothing is silently dropped — the PS asks for exactly this.

Honest weakness: `NO_MATCH` rejection is only **8.3% (1 of 12)** — most garbage lands in `REVIEW` rather than being refused outright. That is the safe failure direction (a human sees it) but it costs review-queue time; tightening it is listed under recommended improvements.

15. Database and data lifecycle

Single-file SQLite via SQLAlchemy (`server/db.py`): `activities`, `linked_events`, `review_queue` (+ clarification columns), `alias_lexicon`, `audit_records`, `jobs`. The lifecycle is ingest → extract → link → (auto-write or review) → audit, and every write is attributable.

- **Append-only audit.** `AuditRecord` rows are never mutated; corrections append. The audit drawer shows every write to an activity: field, old→new, source file and exact line/row, confidence, auto vs planner-confirmed. Where no single line applies it says “no single line” rather than inventing one (DEMO.md §4).
- **Demo reset that survives schema change.** `scripts/reset_demo.py` clears rows (no file deletion, works while the server runs) and rebuilds the schema when model columns differ — SQLAlchemy's `create_all` never adds missing columns, a mid-demo failure mode caught during testing.
- **Integrity rules.** No Actual Start after the data date (hard block); no Actual Finish before Actual Start (hard block); predecessor-not-started warns without blocking (`server/main.py`).

16. Security architecture

Stated plainly: **the MVP has no authentication, no TLS, no multi-tenancy**. Any caller can POST /ingest or resolve a review item; the API trusts its network. This is scored honestly at 4.5/10 in the judge analysis (risk R9) and is the first production item. What the MVP does provide is *integrity machinery* that production security will stand on: append-only audit, sha256 dedup, per-write provenance, hard date-integrity blocks, and an extension allow-list on upload.

17. Prompt-injection protection

NAVIS's defence is architectural: untrusted field text is processed by regex that cannot be talked out of its patterns, and the LLM — when enabled — is a bounded, read-only intent classifier whose output is re-validated before it may touch state.

- The LLM output schema asks for **no dates and no tags-as-truth**; both stay regex-owned (extraction/llm_backend.py).
- Pydantic validation with extra='forbid'; enum-constrained discipline/status so a jailbroken output cannot inject out-of-vocabulary values.
- Every LLM value is re-validated by the same deterministic parsers before use.
- Voice-agent path: bounded timeout (5 s, one attempt, no retry), silent fallback that keeps all collected slots (server/agent_llm.py); a stalled executor is shut down without waiting — a bug found and fixed in testing.
- Slot parsing accepts only validated choices; unknown values fail tests, not the demo (discipline_label() raises on unknown enum).

Residual risk, stated

A crafted DPR could still steer intent classification (e.g. mislabel a discipline). Because the fields that reach the schedule are regex-owned and every write is audited with provenance, the worst case is a wrong REVIEW suggestion — visible, attributable and reversible, never a silent schedule corruption.

18. Audit trail

Every actual-date write appends an immutable AuditRecord: which field changed, the old value, the new value, the source file **and the exact line or spreadsheet row**, the confidence at write time, and whether it was applied automatically or confirmed by a planner. A worked example from a real run: confirming one item produced four audit records on PIP-SPL-1028, three citing piping_progress.xlsx row 10. The append-only property is stated in the UI itself.

19. Institutional memory

The PS names institutional memory a co-equal outcome, and NAVIS treats it as primary: GET /memory/query computes, from captured execution data alone — planned vs actual duration by activity type (worst overrun first), slip by discipline, recurring delay causes with occurrence counts and days lost, and a suggested duration for future planning (requires ≥ 2 completions; e.g. PIP-HYT: baseline 5 d, suggested 6 d, based on 3 completed of 5).

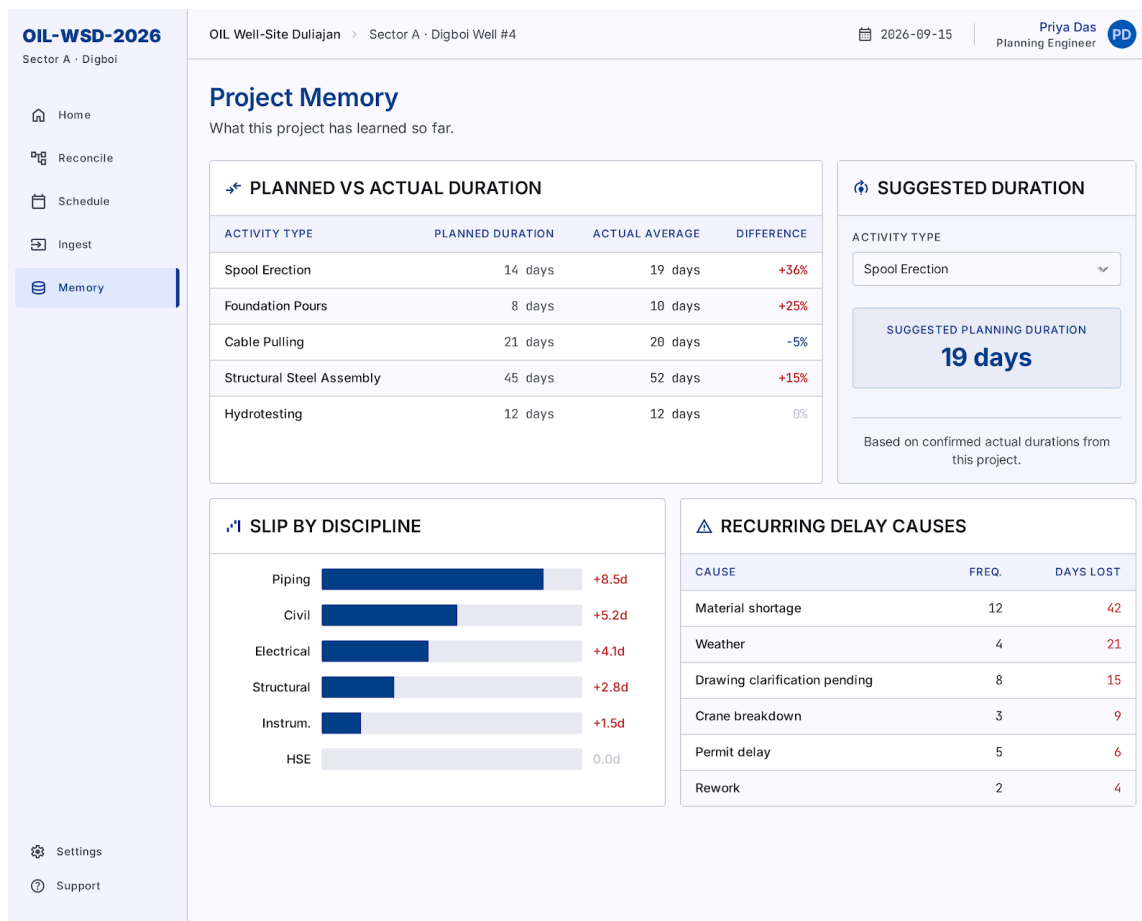


Figure 6. Planning engineer — Memory screen: execution patterns become planning inputs. Design mockup; the shipped screen shows the same four panels with sample sizes visible.

Sample size, shown not hidden

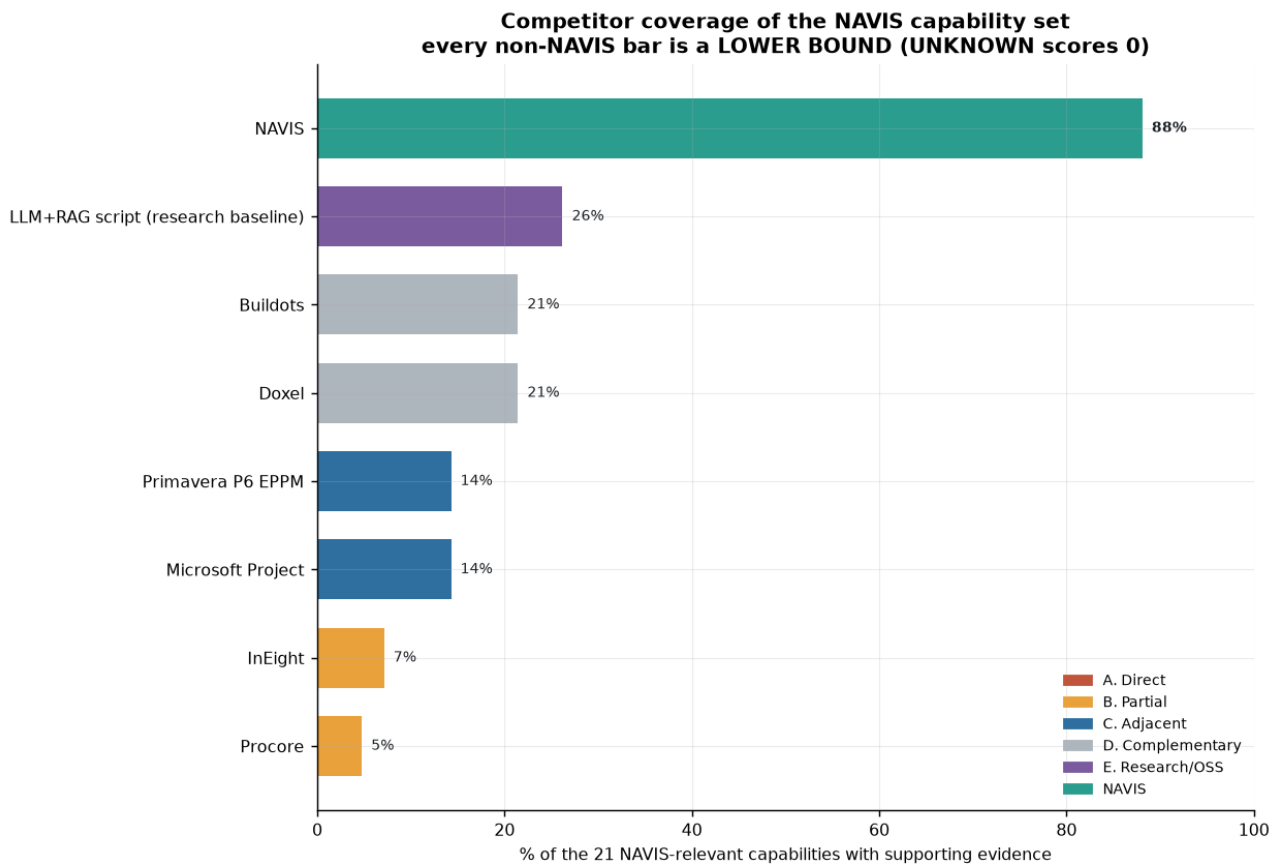
47 of 120 activities have both an actual start and finish. Every memory panel prints its own *n* and says “no completed work yet” in words rather than drawing an empty bar. The architecture is the contribution; the sample grows with every real ingest.

20. Analytics

The clean, discipline-tagged, provenance-bearing event dataset serves the PS's purpose (a): live input for performance analytics. Implemented today: duration overruns by activity type, productivity by discipline, keyword-matched delay causes with days lost, quantity-based % complete per node, and cross-source conflict surfaces. Forecasting / risk-pattern discovery is future work — the dataset design anticipates it but no forecasting model exists in the repo.

21. Competitor analysis

Competitor cells are **rubric scores from published evidence, not vendor code audits** (evidence confidence LOW for every non-NAVIS row; UNKNOWN scores 0, so each non-NAVIS bar is a lower bound). The positioning conclusion survives the caveat: the plan side (P6, MS Project) and the capture side (Procore, InEight) leave the linking layer — prose/voice to L5/L6 with provenance, confidence and audit — structurally unaddressed in published material. Vision competitors (Buildots, Doxel) are complements, not rivals (risk R12).



RUBRIC + published evidence - scoring defined in research/COMPETITIVE_LANDSCAPE.md; cells in research/competitor_matrix.csv. YES=1, PARTIAL=0.5, NO/UNKNOWN=0.

Figure 7. Competitor coverage of the 21 NAVIS-relevant capabilities (RUBRIC + published evidence; scoring method and honesty statement in research/COMPETITIVE_LANDSCAPE.md). Take-away: NAVIS's niche is the linking layer between capture tools and planning tools — not a replacement for either.

22. Innovation analysis

Feature-by-feature, self-assessed on four axes (RUBRIC — reasoned judgement against the code audit, not measurement). The chart is sorted by defensibility: the top of the chart is what to claim; the bottom is what *not* to claim.

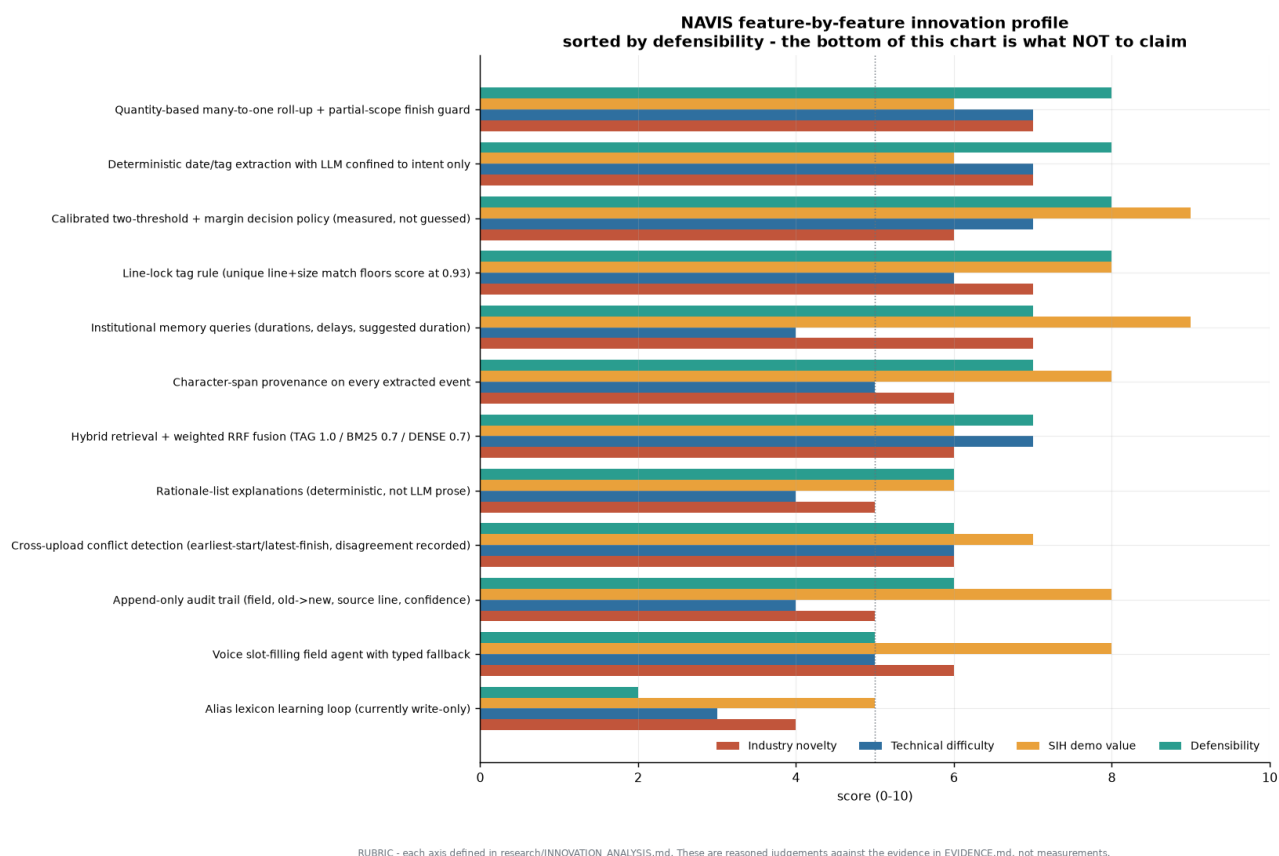


Figure 8. Innovation profile (RUBRIC; defined in research/INNOVATION_ANALYSIS.md). Take-away: the defensible core is the measured decision policy + line lock + deterministic-extraction architecture; the alias learning loop (write-only today) is scored 2/10 and should never be claimed as 'self-learning'.

23. Technical defensibility

What makes the engineering survive cross-examination: every headline number is reproducible from a read-only harness against the real engine and dataset (research/data/*.py); every design decision has a stated reason (ARCHITECTURE.md opens with 'five places I think you're wrong'); and the two known defects — the dense-feature renormalisation and the write-only lexicon — are documented with measurements rather than hidden (densefix.json: fixing the defect is worth +2.5 pts Top-1 and +1.6 pts coverage at unchanged 100% precision).

Claim a judge might challenge	Defence	Evidence
"Why not just BM25?"	BM25 ranks better but gates worse: at 100% auto-precision the hybrid covers 50.4% vs ~35% for BM25-only	bm25gate.json + ablation.json
"Why not just an LLM?"	LLM never writes schedule fields; output re-validated; regex owns dates/tags	llm_backend.py, test_llm_guards.py
"Your eval is circular"	conceded; tag-stripped subset (n=87) reported: Top-1 70.1% without the decisive tag	ablation.json, EXPERIMENTS.md
"There's a defect in scoring"	yes — measured, documented, fix benchmarked	densefix.json
"Coverage of 50% seems low"	it buys 100% precision; coverage-vs-precision is a measured dial, and review items cost ~10 s	eval_output.txt curve

Table 6. Pre-answered challenges. Full drill in research/JUDGE_QUESTIONS.md.

24. PS requirement coverage

All 26 requirements derived from the PS text are audited against code in `research/feature_gap_matrix.csv` (statuses read from code, not documentation): 16 IMPLEMENTED, 5 PARTIAL, 3 MISSING (OCR, P6/MSP import, authentication), 2 NOT_REQUIRED_FOR_MVP. Every PS Expected-Outcome bullet has at least a partial, demonstrable answer.

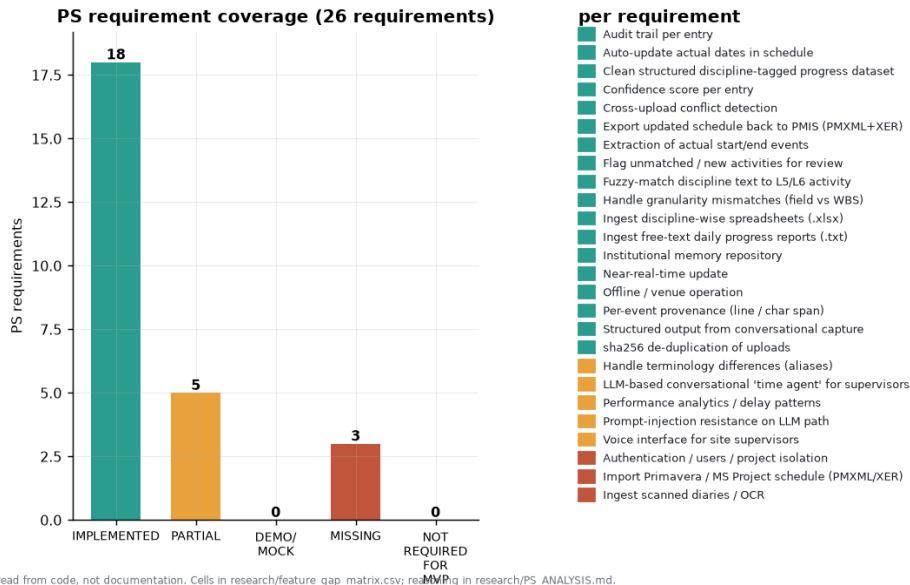
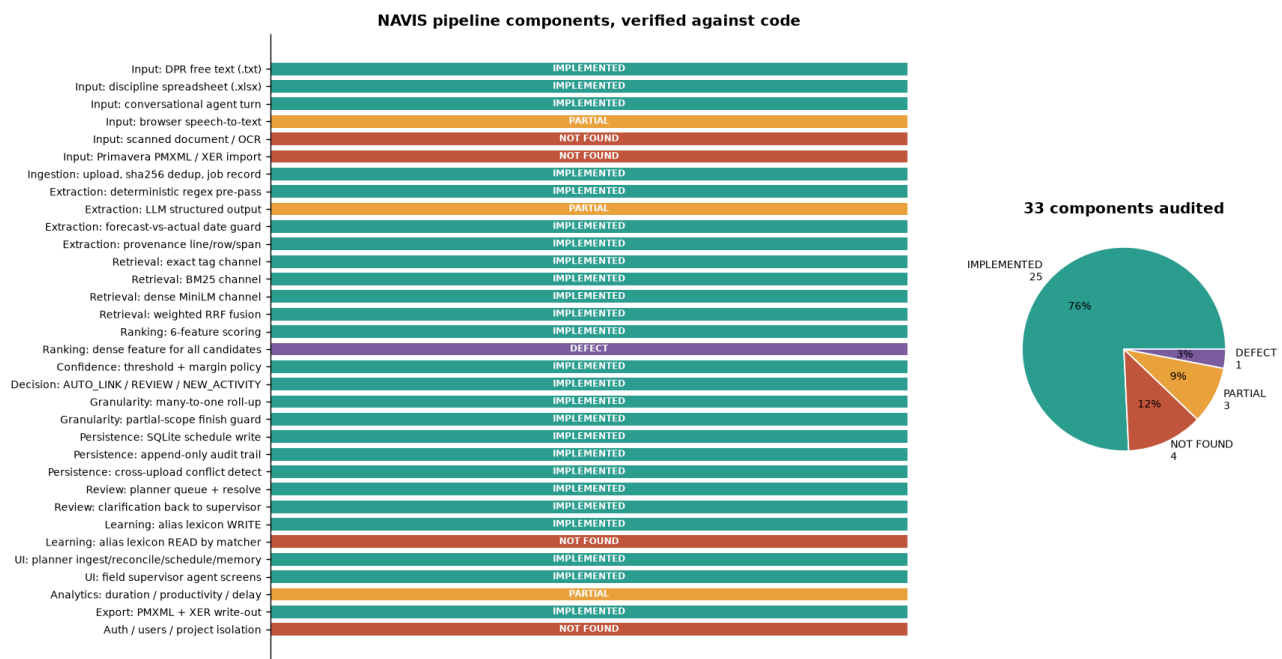


Figure 9. PS requirement coverage (AUDITED — statuses read from code; cells in `research/feature_gap_matrix.csv`, reasoning in `research/PS_ANALYSIS.md`). Take-away: the red bars are exactly what to concede to judges before they find them.

25. Current MVP completeness

34 pipeline components audited component-by-component (`research/NAVIS_TECHNICAL_AUDIT.md`): 24 IMPLEMENTED, 6 PARTIAL, 1 DEFECT (dense-feature renormalisation — measured and benchmarked), 3 NOT FOUND (OCR, PMXML/XER import, auth). The defects and gaps are part of the record.



AUDITED - every row traced to a file and symbol in research/NAVIS_TECHNICAL_AUDIT.md. DEFECT = implemented but measurably wrong (see research/EXPERIMENTS.md).

Figure 10. Implementation status of every pipeline component, verified against code (AUDITED). Take-away: the MVP's core spine — ingest → extract → retrieve → score → decide → roll-up → persist → audit → review — is fully implemented; the gaps sit at the edges (OCR, schedule import, auth, closed learning loop).

26. Scalability

Measured only at demo scale — nothing beyond that is claimed. 120-activity schedule, 266 events in 1.86 s (~7 ms/event), 4.4 s one-time cold start for index + embedding. The architecture scales linearly in schedule size for retrieval (BM25 index + one embedding pass) but **brute-force cosine similarity and SQLite bound the ceiling**: no vector index (FAISS/pgvector), no background workers, single-process. Real projects run 5k–50k L5/L6 activities; a vector index, Postgres and an async ingest queue are the identified scaling path.

27. MVP architecture vs production architecture

Concern	MVP (implemented)	Production (proposed)
Schedule store	SQLite, single project	PostgreSQL, multi-project with RLS
Schedule import	hand-written JSON baseline	PMXML/XER/MS Project parsers; P6 web services
Auth	none	SSO + role-based access; field role scoped to own discipline
Retrieval	brute-force cosine, in-process MiniLM	pgvector/FAISS index; batched encoder service
Ingest	synchronous POST	job queue + workers; SLA-monitored reprocessing
LLM	optional, off by default, bounded 5 s	managed gateway with cost controls; per-tenant keys
Learning loop	alias lexicon written, never read	lexicon read at retrieval; correction retraining
Security	trust-the-network	TLS, auth, rate limits, tenant isolation; audit trail already append-only
Deployment	laptop (venv + npm)	containers; offline site deployment with local model cache

Table 7. Honest MVP-to-production gap map. The integrity spine (provenance, audit, confidence) carries forward unchanged.

28. Experiments / evaluation results

All results measured with read-only harnesses against the real engine and dataset/ground_truth.csv (254 labelled mentions; 242 gold-positive, 12 NO_MATCH). Reproduce with research/data/*.py and eval.py; full log in research/EXPERIMENTS.md.

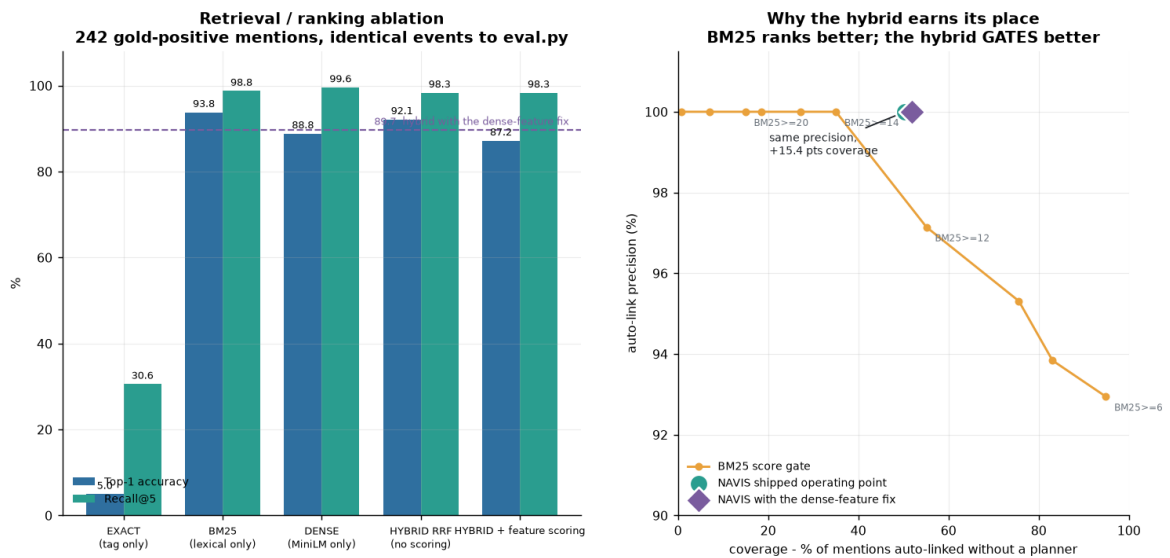


Figure 11. Left: retrieval/ranking ablation on 242 gold positives (MEASURED). Right: the BM25-only gate curve vs the shipped hybrid operating point and the dense-feature fix (MEASURED). Take-away: BM25 ranks better; the hybrid GATES better — at 100% auto-link precision the hybrid covers 50.4% of mentions vs ~35% for BM25-only, and the fix adds a further point of coverage at unchanged precision.

Metric	Value	Detail
Top-1 accuracy	87.2%	211/242 gold positives
Precision (suggestions)	83.1%	207 correct suggestions
Recall (suggestions)	85.5%	of 242 gold positives
AUTO_LINK coverage	50.4%	128 correct auto-links / 254 mentions
Auto-link precision	100.0%	zero wrong auto-links in the confusion table
NO_MATCH rejection	8.3%	1/12 correctly refused — known weakness
Review-queue load	121 items	~10 s each for the planner
Tag-stripped Top-1	70.1%	n=87, matching without the decisive tag signal
Latency	1.86 s / 266 events	~7 ms/event + 4.4 s one-time cold start

Table 8. Headline evaluation results (MEASURED — research/data/eval_output.txt).

29. Weaknesses and limitations

Twelve risks are plotted below (RUBRIC — probability × impact judgement; each risk traces to a finding in the technical audit or judge-question drill). The four highest-severity: no authentication (R9), the hand-written baseline instead of a parsed Primavera file (R2), the measured dense-feature defect (R3), and circular evaluation (R4).

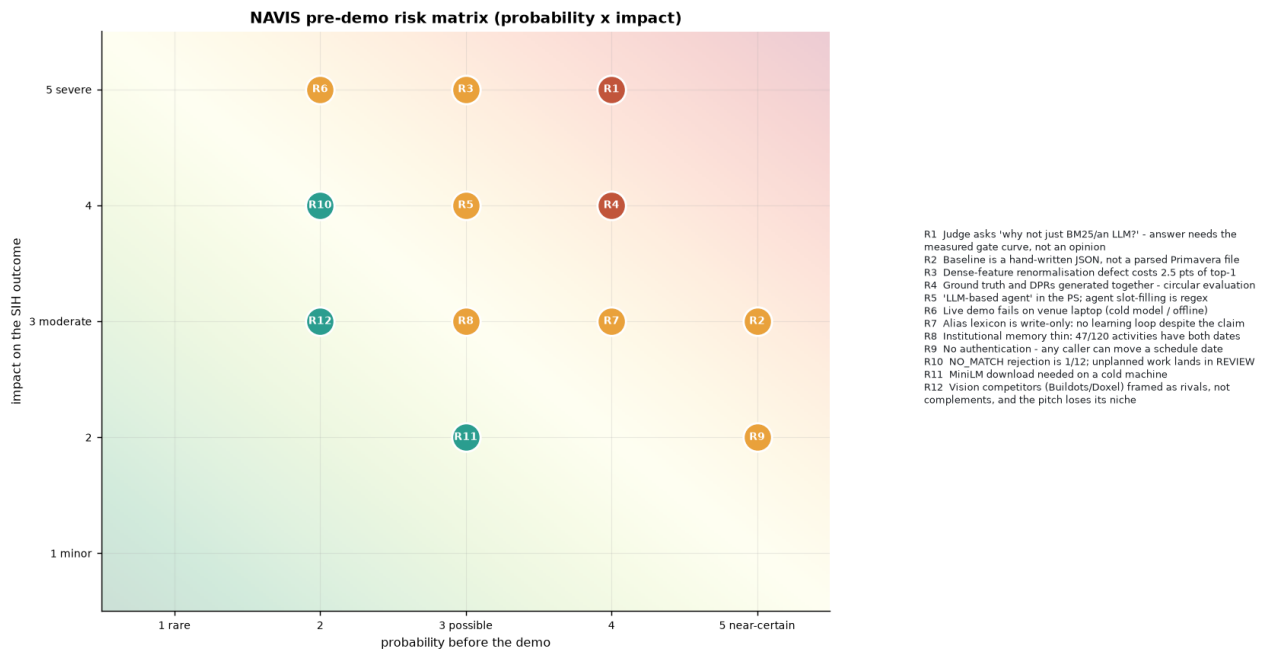


Figure 12. Pre-demo risk matrix (RUBRIC — this analysis's judgement). Take-away: the two red-zone risks (R1 'why not just BM25/an LLM?', R9 no auth) are exactly the ones answered with measurements in §23 and conceded in §16.

- **Data caveats.** Ground truth and DPRs generated together — circular evaluation conceded; 47/120 activities with both actual dates bound the memory analytics.
- **Feature caveats.** Alias lexicon write-only (R7); NO_MATCH rejection 8.3% (R10); agent dialogue deterministic, LLM optional (R5).
- **Platform caveats.** No auth (R9); SQLite single-process; MiniLM download needed once on a cold machine (R11); browser-ASR dependent on venue browser (mitigated by typed fallback, verified end-to-end).

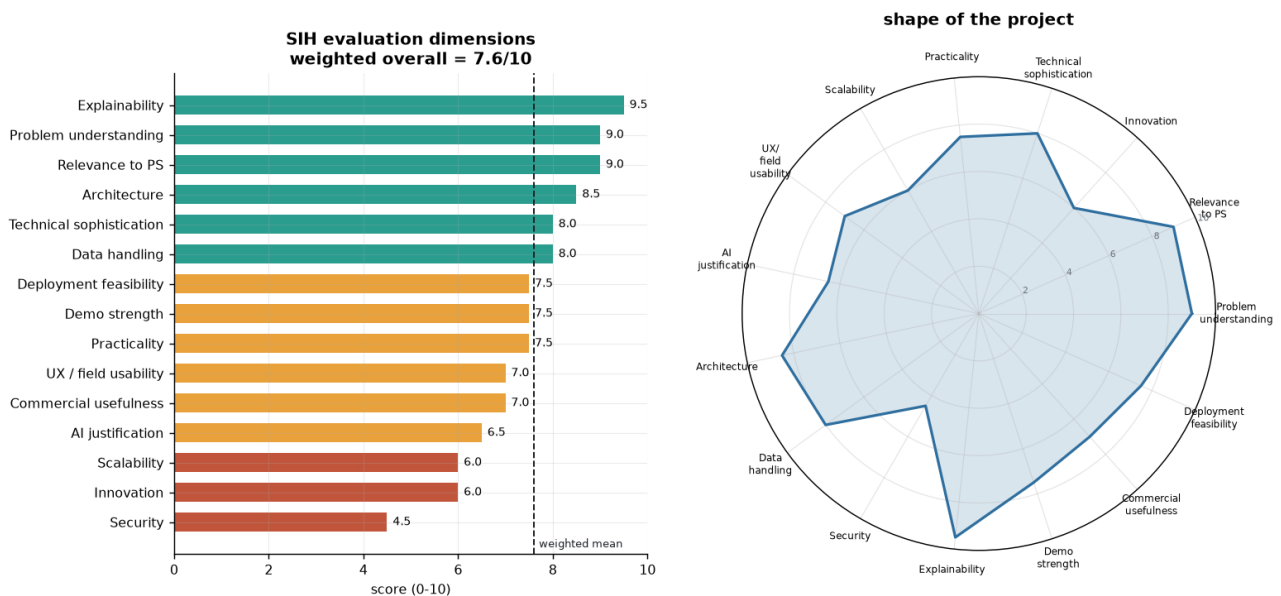
30. Recommended improvements

Priority	Improvement	Why
1	Ship the dense-feature fix (benchmarked: +2.5 pts Top-1, +1.6 pts coverage)	already measured; smallest effort, clearest gain
1	Read alias_lexicon back in retrieval — close the learning loop	converts the write-only gap into the PS's learning claim
2	PMXML/XER schedule import	removes the hand-written-baseline objection (R2) at its root
2	Authentication + project isolation	R9; prerequisite for any real deployment
3	Independent human relabelling of ground truth	kills the circularity objection (R4) with data, not words
3	Vector index + Postgres + async ingest	the identified scaling path beyond demo scale
4	Tighten NO_MATCH rejection beyond 8.3%	saves review-queue time; keeps the safe failure direction
4	On-device ASR for noisy-site voice	browser ASR is the field-UX ceiling today

Table 9. Prioritised improvements, ordered by measured impact per effort.

31. SIH judge analysis

A 15-dimension self-scorecard (RUBRIC — this analysis's judgement, not an official SIH rubric). Weighted overall 7.6/10. Strategy: lead with explainability (9.5), architecture (8.5) and PS relevance (9.0); concede security (4.5), scalability (6.0) and innovation (6.0) before judges find them — conceding with a measurement converts a weakness into credibility.



RUBRIC - scores and weights are this analysis's judgement, defined with reasoning in research/SIH_JUDGE_ANALYSIS.md. Not a measurement and not an SIH rubric.

Figure 13. SIH evaluation dimensions, self-scored (RUBRIC; defined in research/SIH_JUDGE_ANALYSIS.md). Take-away: the shape is broad with two deliberate dips — security and scalability — both owned and both answered.

32. Demo strategy

The demo path is rehearsed end-to-end (DEMO.md): two terminals, reset to a known state with `scripts/demo_reset.ps1` (~3 s; expects the summary line 120 activities / 67 with actuals / 118 review items), then: Home conflict cards → Ingest → Reconcile (confirm one item, open the audit drawer showing four appended records on PIP-SPL-1028) → Schedule (confidence badges) → Memory (sample sizes in words) → Field voice agent (?view=field).

- **Transcript review is the point, not an obstacle:** ASR mangles tags like 24"-P-1001-A1A, and tag overlap is the matcher's strongest feature — edit, then send.
- **Mic failure is a normal state:** the screen shows "Microphone unavailable / Typing works just as well", the text input becomes primary, no red on screen. A typed-only session was verified end-to-end to a matched activity and review item.
- **The LLM is off by default** and cannot break the demo: no client built, no socket opened when `EXTRACTION_PROVIDER=rules`.
- Recovery playbook on stage: error banner = API port mismatch; empty panel = by design (independent fetches); stale port = restart on 8001 with `VITE_API_URL`.

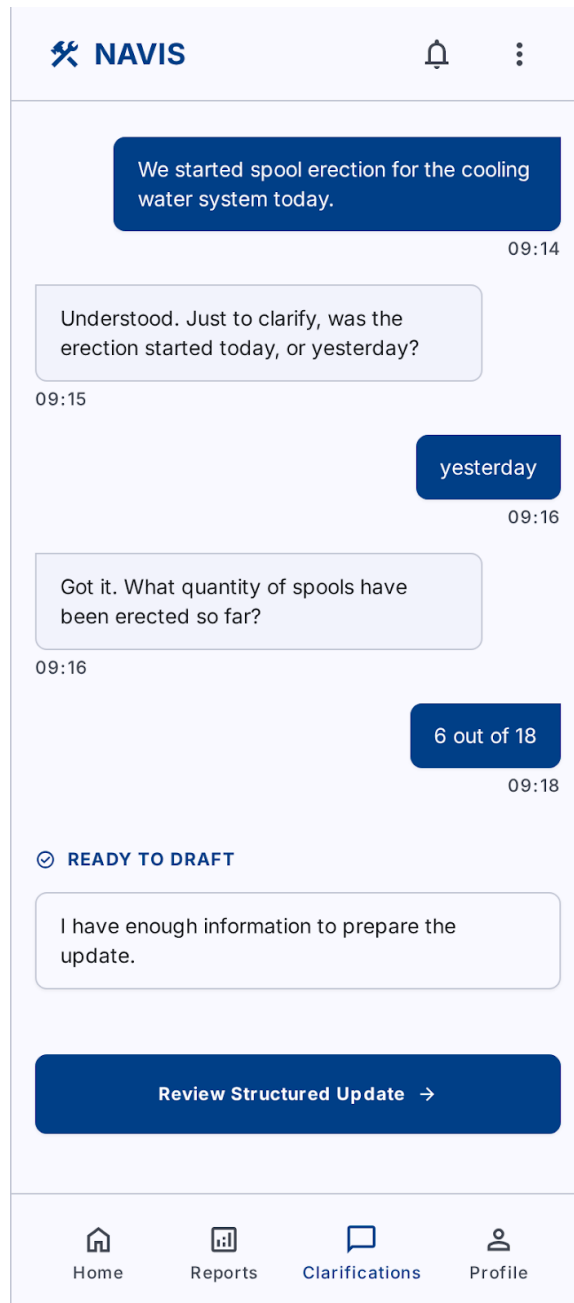


Figure 14. Field supervisor — the voice agent asks for one slot at a time (date, then quantity: “6 out of 18” kept as completed 6 / planned 18) and hands over to a structured card. Design mockup of the shipped flow (frontend/src/pages/Field.tsx).

33. Difficult judge questions and answers

Condensed from research/JUDGE_QUESTIONS.md — every answer cites repository evidence, and each is backed by a measured number rather than an opinion.

Judge question	Answer (evidence-backed)
"Why not just BM25 — or just hand everything to an LLM?"	Measured: BM25-only covers ~35% at 100% gate precision vs 50.4% for the hybrid, zero wrong auto-links (bm25gate.json). The LLM never decides: output re-validated, dates/tags regex-owned.
"Your schedule is a JSON file, not a real Primavera export."	Correct and conceded. PMXML/XER exist as export only; import is the first post-hackathon item (risk R2).
"There's a defect in your ranking?"	Yes — the dense feature is missing for candidates absent from the dense channel. Fix measured at +2.5 pts Top-1 (densefix.json); documented, not rushed.
"Your evaluation is circular."	Conceded; mitigated by 87 tag-stripped mentions (Top-1 70.1% reported, not hidden). Independent relabelling recommended before external claims.
"The PS says LLM-based agent. Yours is regex?"	Slot-filling is deterministic on purpose — it can't fail on stage; the LLM path exists behind a flag with a 5 s timeout and silent fallback.
"What if the demo laptop is offline?"	Defaults are offline-first: rules provider, local MiniLM (local_files_only), typed fallback verified end-to-end.
"Show the learning loop."	Corrections write alias_lexicon rows today; the matcher doesn't read them yet — scored 2/10 defensibility rather than claimed.
"Institutional memory from 47 activities?"	Sample sizes are shown on every panel; suggested duration requires ≥ 2 completions. The architecture is the contribution.
"Where is auth?"	There is none in the MVP — named limitation, first production item; the append-only audit trail is what production security will stand on.
"What happens to unplanned work?"	NEW_ACTIVITY surfaces it to the planner. NO_MATCH rejection is 8.3% — garbage lands in REVIEW (the safe direction), not in the schedule.
"Buildots/Doxel already do this."	They are camera-based complements; NAVIS links what supervisors say and write — positioning is the linking layer (R12).

Table 10. The hard-question drill (full 12 in research/JUDGE_QUESTIONS.md).

34. Final verdict

NAVIS is a complete, honest, measured answer to PS 26122's core: the full ingest → extract → link → reconcile → remember spine works end-to-end on the recommended corpus, its matching policy is calibrated to a measured 100% auto-link precision, its audit trail makes every write defensible, and its institutional-memory layer covers the PS outcome most teams will skip. Its known gaps — no auth, no schedule import, a write-only learning loop, demo-scale only — are documented with the same rigour as its results.

Where this wins or loses

Wins on: measured linking quality with zero wrong auto-links; explainability and audit; institutional memory; offline practicality. Loses or draws on: raw innovation of individual components (hybrid retrieval is table stakes), scale beyond demo, and security. The strongest card is that every number in this report can be re-run in front of a judge: python eval.py.

35. The strongest one-line description of NAVIS

NAVIS turns what a site supervisor says, types or reports into audited schedule fact — linking every field mention to the right L5/L6 activity with measured 100% auto-link precision, and turning every project into queryable institutional memory.

Shorter still: “**The field talks. The schedule listens. Every word is audited.**”

Appendix — evidence index

Every figure and table in this report carries an evidence label:

MEASURED — read from research/data/*.json or eval.py output, produced by read-only harnesses against the real engine and dataset/ground_truth.csv.

AUDITED — component/requirement statuses traced to a file and symbol in research/NAVIS_TECHNICAL_AUDIT.md and the matrix CSVs.

RUBRIC — reasoned judgement (SIH scorecard, risks, innovation profile, competitor cells), labelled as such on the figure itself and never presented as measurement. Competitor cells use published evidence only, evidence confidence LOW, UNKNOWN scores 0 — every non-NAVIS bar is a lower bound.

Supporting documents: research/EXPERIMENTS.md · research/JUDGE_QUESTIONS.md ·
research/PS_ANALYSIS.md · research/COMPETITIVE_LANDSCAPE.md ·
research/INNOVATION_ANALYSIS.md · research/SIH_JUDGE_ANALYSIS.md · research/EVIDENCE.md.